

MPI Sinkhorn Optimal Transport Solver

Row-wise Data Decomposition on a Physical VM Cluster

Nguyễn Tất Thanh Đặng Thịnh Tường Minh Nguyễn Ngọc Minh Kiều Sơn Tùng

Hanoi University of Science and Technology
School of Information and Communication Technology

2026



SOICT

Project Goal and Deliverables

Assignment requirements

- ▶ C/C++ MPI implementation
- ▶ At least three physical machines
- ▶ Explain parallelism, decomposition, mapping, communication
- ▶ Validate correctness against sequential baseline
- ▶ Measure runtime, speedup, efficiency, and load balance

Delivered

- ▶ C++17 Sinkhorn solver
- ▶ Sequential and MPI modes
- ▶ Router/Ethernet and Wi-Fi cluster results
- ▶ JSON/CSV benchmark logs
- ▶ Dynamic scheduling extension

Problem: Entropy-Regularized Optimal Transport

Optimal transport finds a transport plan P from source distribution a to target distribution b .

$$K_{ij} = \exp(-C_{ij}/\varepsilon)$$

$$u = a \oslash (Kv), \quad v = b \oslash (K^T u)$$

$$P = \text{diag}(u)K\text{diag}(v)$$

Why this is parallelizable

- ▶ Dense $N \times N$ kernel matrix
- ▶ $O(N^2)$ work per iteration
- ▶ Row updates are independent
- ▶ Column update requires global reduction

Parallel Algorithm

Data decomposition

- ▶ Split rows contiguously by MPI rank
- ▶ Rank r owns $K_r = K[s_r : e_r, :]$
- ▶ Vector v is replicated on all ranks
- ▶ Each rank updates local u_i

Communication

$$\text{local_col}_j = \sum_{i=s_r}^{e_r-1} K_{ij} u_i$$

$\text{global_col} = \text{MPI_Allreduce}(\text{local_col})$

- ▶ One length- N Allreduce per iteration
- ▶ Main bottleneck at high process counts

MPI Sinkhorn Pseudocode

```
Build local rows [s_r, e_r)
Build local dense kernel K_r
Initialize local u and replicated v

for iter = 1..max_iters:
    for each local row i:
         $u[i] = a[i] / \sum_j K_r[i,j] * v[j]$ 

    local_col[:] = 0
    for each local row i:
        for each column j:
             $local\_col[j] += K_r[i,j] * u[i]$ 

    global_col = MPI_Allreduce_sum(local_col)
    for each column j:
         $v[j] = b[j] / global\_col[j]$ 
```

Cluster and Hardware

Node	Physical CPU	C/T	VM CPU	VM RAM	GB6 score	Role
Master	Intel Core i5-9300H	4/8	4 vCPU	4 → 8 GB	1291 SC / 3737 MC	Master for router and latest Wi-Fi runs
Worker 1	Intel Core i5-9300H	4/8	4 vCPU	4 → 8 GB	1291 SC / 3737 MC	Router/Wi-Fi worker
Worker 2	AMD Ryzen 5 5600H	6/12	4 vCPU	3 → 6 GB	1594 SC / 5716 MC	Router worker, stronger CPU but initially memory-limited
Worker 3	Intel Core i3-1115G4	2/4	4 vCPU	4 → 6 GB	1571 SC / 3056 MC	Latest Wi-Fi worker

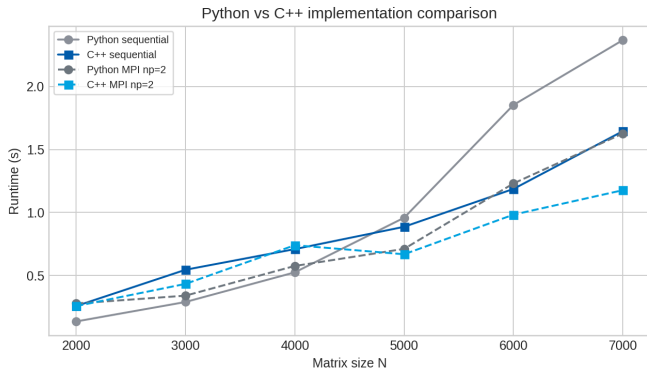
- ▶ Router: TP-Link Archer C20 AC750
- ▶ 2.4 GHz: 300 Mbps, 5 GHz: 433 Mbps
- ▶ Four 10/100 Mbps LAN ports



Benchmark Plan

- ➊ **Input-size calibration:** $N = 2000..12000$, fixed 50 iterations
- ➋ **Long workload:** $N = 8000$, increase iterations to reach 2–3 minutes
- ➌ **Process sweep:** vary MPI processes at fixed $N = 8000$
- ➍ **$2N$ speedup:** run $N = 16000$ with `np=1,2,4,8`
- ➎ **Network comparison:** router/Ethernet, Wi-Fi, phone hotspot
- ➏ **Dynamic scheduling:** contiguous, weighted chunks, runtime queue

Python Prototype vs C++ Implementation

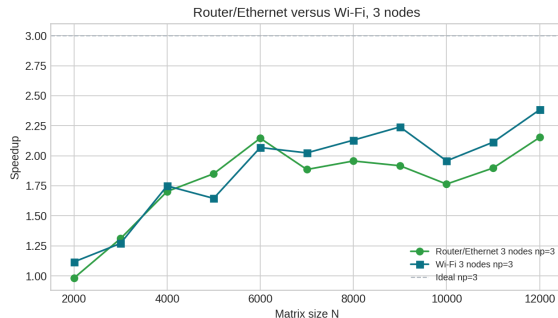
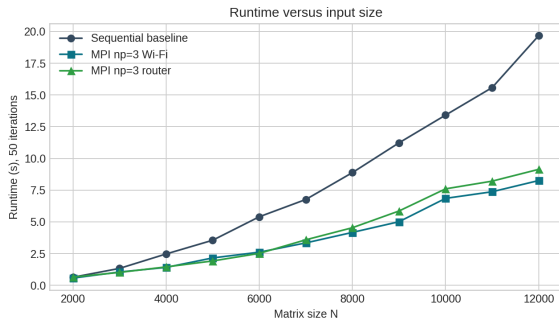


Why this slide matters

- ▶ The first MVP was written in Python/mpi4py for correctness.
- ▶ The final submission path is C++/MPI to satisfy the course requirement.
- ▶ C++ becomes more attractive as the benchmark grows and avoids Python runtime overhead.

Dataset: two-node benchmark, 3-run averages, sizes 2000–7000.

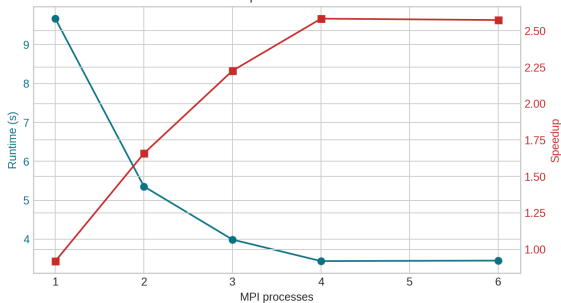
Input-Size Sweep: Router vs Wi-Fi



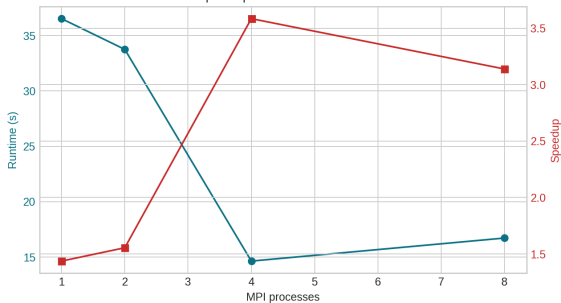
- ▶ MPI becomes useful when N is large enough.
- ▶ At $N = 12000$, Wi-Fi three-node run reaches $2.383\times$ speedup.
- ▶ Best observed router three-node point: $2.8859\times$ speedup, 0.962 efficiency.

Process Sweep and $2N$ Speedup

Process sweep at $N=8000$ over Wi-Fi

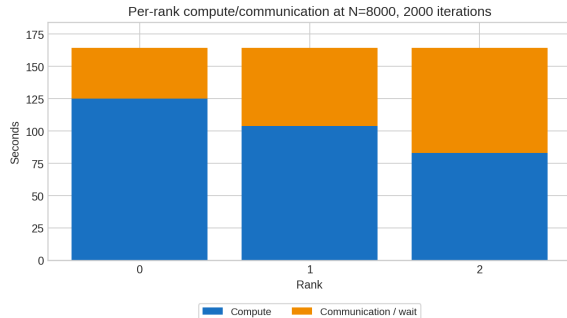
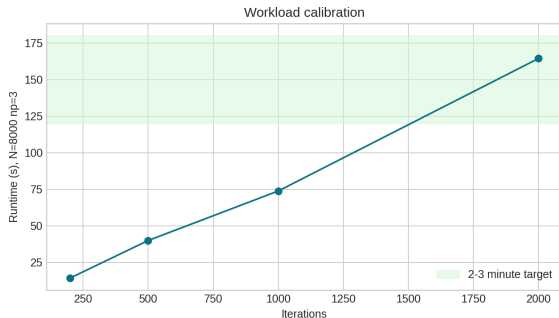


$2N$ speedup at $N=16000$ over Wi-Fi



- ▶ At $N = 8000$, $np=4$ is better than $np=6$.
- ▶ At $N = 16000$, $np=4$ reaches $3.585\times$ speedup.
- ▶ $np=8$ is slower than $np=4$: communication dominates.

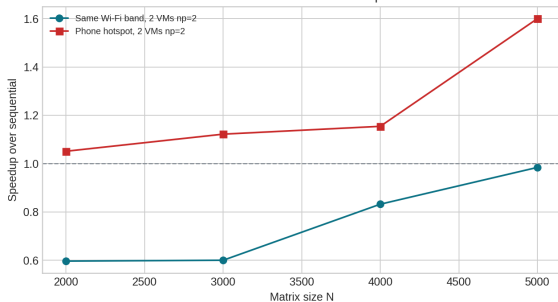
Calibration and Load Balance



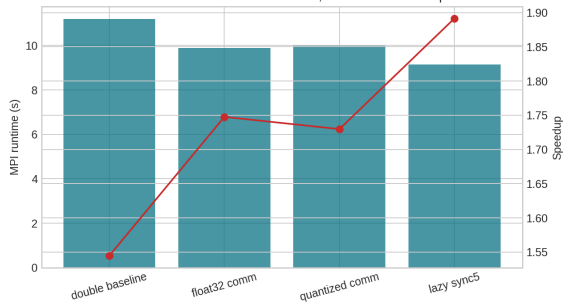
- ▶ $N = 8000$, 2000 iterations gives 164.394 s: within the 2–3 minute target.
- ▶ Total rank times are balanced, but compute/communication split reveals node heterogeneity.

Network and Communication Experiments

Two-node wireless network comparison



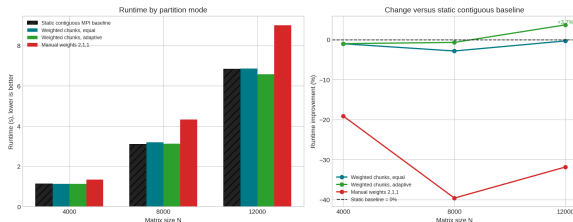
Communication variants at $N=12000$, same-band Wi-Fi np=2



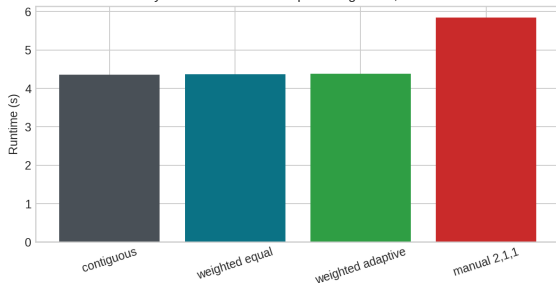
- ▶ Wireless topology matters: same-band Wi-Fi and phone hotspot behave differently.
- ▶ Lazy sync every 5 iterations gives the best communication-variant speedup at $N = 12000$.
- ▶ Float32 communication halves payload and is the safest compression-style optimization.

Dynamic Scheduling

Dynamic scheduling compared with static contiguous MPI baseline, router 3 nodes np=3



Dynamic modes under np=6 congestion, N=8000



- ▶ Static contiguous is the MPI baseline: distributed rows, fixed ownership.
- ▶ Adaptive weighted chunks improve large $N = 12000$ by about 3.7% over that baseline.
- ▶ Manual weights can hurt if the assumed machine speeds are wrong.
- ▶ Runtime queue is useful for demonstration but too slow for performance.

Conclusions

- ▶ The C++/MPI Sinkhorn solver satisfies the project requirements and runs on a real physical-machine VM cluster.
- ▶ Row-wise data decomposition gives clear speedup for sufficiently large dense workloads.
- ▶ Main bottleneck: one length- N MPI_Allreduce per iteration.
- ▶ Best stable demo recommendation:
 - Router/Ethernet three-machine setup
 - $N = 8000$, 2000 iterations for a 2–3 minute workload
- ▶ Dynamic weighted scheduling is a promising extension, but static contiguous partitioning remains the reliable baseline.

Backup: Key Raw Result Locations

- ▶ `results/bench_cpp_router3_25_26_75_fixed50_reps4`
- ▶ `results/bench_cpp_router3_report_suite_safe`
- ▶ `results/bench_cpp_wifi3_25_26_190_report_suite`
- ▶ `results/bench_cpp_wifi3_25_26_190_supplemental`
- ▶ `results/bench_cpp_router3_dynamic_fixed50_reps4`
- ▶ `results/bench_cpp_vm2_sameband_variants_fixed50`